BIOS 735: Tree Based Classifiers

Alexander McLain

September 25, 2025

Outline

Introduction to CART

How to grow a Regression tree

How to grow a Classification tree

A Motivating Example

- Suppose we want to predict whether patients are at high risk of complications after surgery.
- Predictors might include:
  - Age
  - BMI
  - Smoking status
  - Blood pressure
- Example:
  - For younger patients, BMI may be the strongest predictor of risk.
  - For older patients, blood pressure might matter more, while BMI is less important.
- This type of **context-specific relationship** is difficult to capture with a regression methods (even with penalized regression).

Limits of Standard Regression

- Regression requires us to specify:
  - Functional forms (linear, quadratic, etc.), though this can be relaxed with GAM models.
  - Interaction terms (e.g., age $\times$ BMI, age $\times$ blood pressure).
- As the number of predictors grows, the number of possible interactions grows rapidly.
- This can lead to:
  - Models that are overly complex and hard to interpret, i.e., the GAM with selection revote on Tuesday.
  - Risk of misspecification if the "true" interaction or nonlinearity is not included.
- In practice, we may miss important associations or build models that don't generalize well.

Why CART?

- Classification and Regression Trees (CART) provide a flexible, non-parametric alternative.
- CART automatically:
  - Detects nonlinear associations.
  - Identifies higher-order interactions, e.g. differential effects in age $\times$ BMI $\times$ smoking status.
  - Adapts splits to different subgroups of subjects.
- No need to pre-specify functional forms or interaction terms.
- This makes CART especially powerful in settings with many predictors and possible context-specific effects.

CART: Basic Idea

- Suppose we have two predictors X_1 and X_2 and we are trying to classify observations into groups (e.g., disease vs. no disease).
- Tree-based methods partition the predictor space into **rectangular blocks**.
- Each block corresponds to a simple rule (e.g., "if $X_1 < 3.5$ and $X_2 \geq 2.1$ then predict diseased").

CART: Basic Idea

- This strategy is nonparametric: we do not assume any particular functional form linking X to Y.

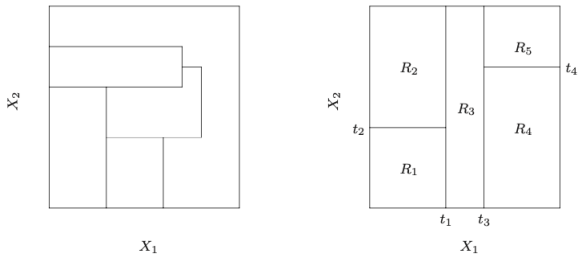
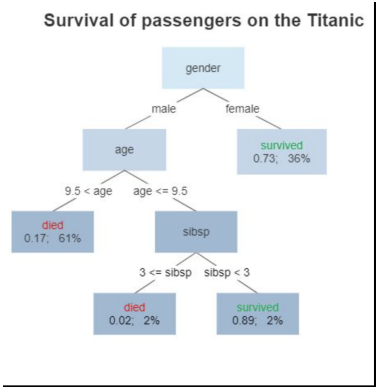


Figure: CART Example from ESL II (page 306): recursive binary splits divide the predictor space into regions.

CART Regions and Predictions

- The results from a CART are usually displayed as a tree.
- The ends of the tree are called the terminal regions or terminal nodes and denoted by R_m.



CART Regions and Predictions

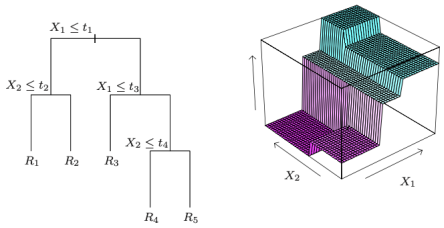


Figure: CART Example from ESL II (page 306). Predictor space is divided into decision regions.

- Each R_m has a constant predicted value denoted by $\hat{c}_m$:
  - For regression: $\hat{c}_m = \text{ave}(y_i \mid x_i \in R_m)$.
  - For classification: $\hat{c}_m =$ most frequent class in R_m.
- **Intuition:** CART approximates the regression function by a set of step functions.

Notation

- Data: N observations (x_i, y_i) with $x_i = (x_{i1}, \ldots, x_{ip})$.
- Goal: partition the predictor space into M non-overlapping regions $R_1, \ldots, R_M$.
- Within each region, the prediction is a constant:

$$f(x) = \sum_{m=1}^{M} c_m I(x \in R_m)$$

  where R_m could be the set of all x with $x_1 < 3.5$, $x_3 > 6.8$, and $x_7 = 1$.
- Every x with be in exactly one R_m.
- If we minimize the sum of squared errors, the best constant is:

$$\hat{c}_m = \text{ave}(y_i \mid x_i \in R_m),$$

  the average y_i for all i with $x_i \in R_m$.

How do we choose the regions R_m?

- Searching over **all possible partitions** of the predictor space is computationally infeasible (the number of possibilities grows exponentially).
- CART uses a **greedy recursive algorithm**:
  1. Start with all data in a single region R.
  2. Consider each predictor j and all possible split points s.
  3. Define candidate regions:

$$R_1(j,s) = \{X \mid X_j \leq s\}, \quad R_2(j,s) = \{X \mid X_j > s\}.$$

     Note: R_1 and R_2 are the child regions, R is the parent region.
  4. Choose (j,s) that result in the biggest decrease in "error."

Splitting: Optimization Problem

- Formally, for squared error loss, CART chooses (j, s) to minimize:

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right]$$

  where

$$\hat{c}_1 = \text{ave}(y_i \mid x_i \in R_1), \quad \hat{c}_2 = \text{ave}(y_i \mid x_i \in R_2).$$

- Practically: scan over predictors and split points $\rightarrow$ pick best.
- Repeat recursively. Each split creates more homogeneous regions.

Further splits

- Suppose that the first split results in

$$R_1 = \{X \mid X_1 \leq 4\}, \quad R_2 = \{X \mid X_1 > 4\}.$$

- For the next step, we consider each predictor j and all possible split points s.

$$R_{11}(j,s) = \{X \mid X_1 \leq 4 \text{ and } X_j \leq s\}, \quad R_{12}(j,s) = \{X \mid X_1 \leq 4 \text{ and } X_j > s\}.$$

or

$$R_{21}(j,s) = \{X \mid X_1 > 4 \text{ and } X_j \leq s\}, \quad R_{22}(j,s) = \{X \mid X_1 > 4 \text{ and } X_j > s\}.$$

- The result of this will be three quantities:
  1. the parent region (R_1 or R_2) where the split occured,
  2. j - the variable being split, and
  3. s - the point we are splitting X_j.

How far to grow?

- If we let the tree keep splitting, we eventually end up with each subject being in their own terminal node.
  - overfit (perfectly predicts training data but has poor generalization).
- Tree size = tuning parameter controlling complexity.
- Common strategy:
  1. Grow a large tree T_0 (stop only when node size is very small, e.g., 5).
  2. Then prune T_0 back to a smaller subtree T using cost-complexity pruning.
- We want a tree that balances fit (low error) and complexity (few leaves).

Error of trees and subtrees

- Subtree T is any tree obtained by pruning T_0 (removing some splits).
- Let $|T|$ be the size of tree T and

$$Q_m(T) = \frac{1}{N_m} \sum_{x_i \in R_m} (y_i - \hat{c}_m)^2, \quad \hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m} y_i$$

  $Q_m(T)$ is the mean squared error (MSE) in terminal node m of tree T.
- The total error (or **Risk**) of tree T:

$$C(T) = \sum_{m=1}^{|T|} N_m Q_m(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{c}_m)^2.$$

Note: $C(T)$ will always go down as the size of the tree $|T|$ increases.

Cost Complexity Criterion

- Minimizing $C(T)$ will lead to overfitting.
- Penalized risk (cost complexity criterion):

$$C_\alpha(T) = C(T) + \alpha|T|$$

  where $|T|$ = number of terminal nodes.
- **Interpretation:** α controls tradeoff between accuracy (low $C(T)$) and parsimony (small $|T|$).
  - large $\alpha \rightarrow$ smaller $|T|$, larger $C(T)$
  - small $\alpha \rightarrow$ larger $|T|$, smaller $C(T)$
- For each α, there is a unique smallest subtree, which we call T_α, that minimizes $C_\alpha(T)$.

Weakest Link Pruning

- How do we prune, i.e., find the best subtree?
- Algorithm: **weakest link pruning**
  1. Start with the full tree T_0.
  2. Iteratively collapse the terminal nodes that increase the error ($C(T)$) the least.
  3. Continue until only the root remains.

Weakest Link Pruning

- How do we prune, i.e., find the best subtree?
- Algorithm: **weakest link pruning**
  1. Start with the full tree T_0.
  2. Iteratively collapse the terminal nodes that increase the error ($C(T)$) the least.
  3. Continue until only the root remains.
- This generates a nested sequence of candidate subtrees, each with its corresponding α value.

$$T_\infty \subseteq \ldots \subseteq T_{\alpha_2} \subseteq T_{\alpha_1} \subseteq T_0$$

where $0 < \alpha_1 < \alpha_2 < \ldots$.

Choosing α

- Now we have a series of α values, and we need to choose the optimal one. How can we do that?

Choosing α

- Now we have a series of α values, and we need to choose the optimal one. How can we do that?
- Use cross-validation to choose $\hat{\alpha} \rightarrow$ select $T_{\hat{\alpha}}$.
- Analogy: similar to regularization in regression (lasso/ridge), but applied to tree complexity.

Classification Trees

- Data: N observations (x_i, y_i) with $y_i \in \{1, 2, \ldots, K\}$.
- Within each terminal node m, estimate class probabilities:

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k).$$

- Prediction rule: assign all observations in region R_m to class

$$k(m) = \arg\max_k \hat{p}_{mk}.$$

- Classification trees thus partition the predictor space and give a majority-vote prediction within each leaf.

Node Impurity Measures

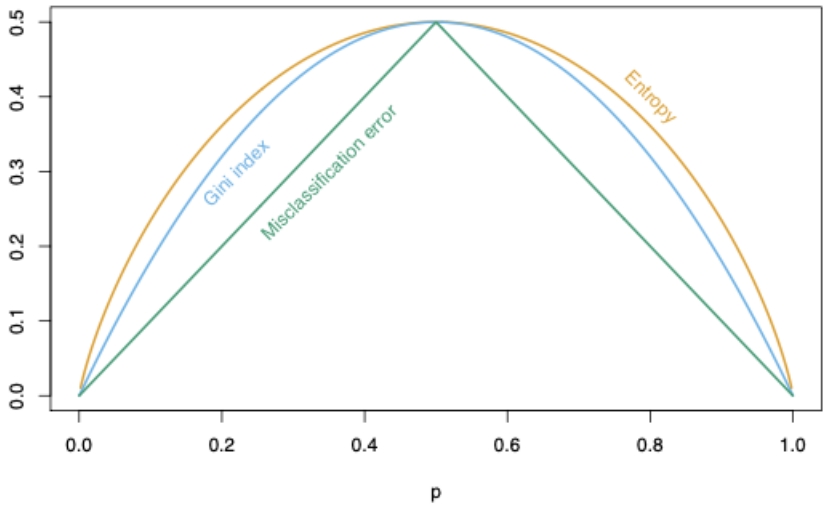
- To decide splits, CART uses node impurity measures:

$$\text{Misclassification error:} \quad 1 - \hat{p}_{mk(m)}$$

$$\text{Gini index:} \quad \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

$$\text{Entropy (deviance):} \quad -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

- Intuition:
  - Misclassification error: fraction misclassified if we always predict majority class.
  - Gini index: measures heterogeneity of class distribution in the node.
  - Entropy: penalizes uncertainty; high values when probabilities are relatively similar
- In practice, Gini or entropy is preferred.


0.0
0.1
0.2
0.3
0.4
0.5
0.0
0.2
0.4
0.6
0.8
1.0
p
Gini index
Misclassification error
Entropy